

Algorithmique et Programmation en C

Diapositives complètes — Dr. Zakaria Sawadogo, École Polytechnique de Ouagadougou

Chapitre 1 — Introduction à l'algorithmique et au langage C

Algorithmique et Programmation en C

DR. ZAKARIA SAWADO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Qu'est-ce qu'un algorithme ?

Un algorithme est une suite finie et non ambiguë d'instructions permettant de résoudre un problème ou d'accomplir une tâche à partir de données d'entrée, en produisant un résultat en un temps fini. Trois propriétés le caractérisent : la **finitude** (il se termine), le **déterminisme** (chaque étape est définie sans ambiguïté) et l'**effectivité** (chaque étape est réalisable).

Avant d'écrire du code, on décrit un algorithme en **pseudo-code**, indépendant de tout langage :

```
ALGORITHME Maximum
ENTRÉE : a, b (deux entiers)
SORTIE : le plus grand des deux
DÉBUT
    SI a > b ALORS
        RETOURNER a
    SINON
        RETOURNER b
    FIN SI
FIN
```

2. Pourquoi le langage C ?

Le C, créé par Dennis Ritchie en 1972, reste incontournable en sécurité informatique car :

- il offre un accès direct à la mémoire (pointeurs), indispensable pour comprendre les vulnérabilités mémoire (buffer overflow, use-after-free, etc.) ;
- une grande partie des systèmes d'exploitation, noyaux et outils bas niveau sont écrits en C ;
- comprendre le C facilite la lecture de code décompilé/désassemblé en analyse de vulnérabilités et en cryptanalyse appliquée.

3. De l'algorithme au programme exécutable

La chaîne de compilation C comporte plusieurs étapes :

1. **Préprocesseur** : traite les directives `#include`, `#define`, `#ifdef`.
2. **Compilation** : traduit le code C en assembleur puis en code objet.
3. **Édition de liens (linking)** : assemble les fichiers objets et les bibliothèques en un exécutable.

```
gcc -Wall -Wextra -std=c11 -o programme programme.c  
./programme
```

4. Premier programme

```
#include <stdio.h>

int main(void) {
    printf("Bonjour, monde !\n");
    return 0;
}
```

- `#include <stdio.h>` : inclut la bibliothèque d'entrées-sorties standard.
- `int main(void)` : point d'entrée du programme ; renvoie un entier au système d'exploitation (0 = succès).
- `printf` : affiche du texte formaté sur la sortie standard.

5. Structure générale d'un programme C

```
directives de préprocesseur
déclarations globales (constantes, prototypes de fonctions)
int main(void) {
    déclarations locales
    instructions
    return code;
}
définitions des autres fonctions
```

À retenir

- Un algorithme se conçoit avant de se coder.
- Le C est un langage compilé, statiquement typé, proche de la machine.
- Toujours compiler avec les avertissements activés (`-Wall -Wextra`) : la majorité des bugs de sécurité en C sont détectables dès la compilation ou l'analyse statique.

Chapitre 2 — Variables, types, opérateurs et entrées-sorties

Algorithmique et Programmation en C

DR. ZAKARIA SAWADO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Variables et types de base

Une variable est un emplacement mémoire nommé, dont le type détermine la taille occupée et les opérations possibles.

Type	Taille usuelle	Exemple	Rôle
<code>int</code>	4 octets	<code>int age = 25;</code>	entier signé
<code>unsigned int</code>	4 octets	<code>unsigned int compteur = 0;</code>	entier non signé
<code>char</code>	1 octet	<code>char lettre = 'A';</code>	caractère / petit entier
<code>float</code>	4 octets	<code>float prix = 19.99f;</code>	flottant simple précision
<code>double</code>	8 octets	<code>double pi = 3.14159265;</code>	flottant double précision
<code>long</code> , <code>short</code>	variable	<code>long total;</code>	variantes de taille de <code>int</code>

La taille exacte dépend de l'architecture et du compilateur : on utilise `sizeof(type)` pour la vérifier, et `stdint.h` (`int32_t`, `uint8_t`, ...) pour des tailles garanties — pratique essentielle en sécurité où les débordements d'entiers sont une classe de vulnérabilité à part entière.

2. Déclaration, initialisation, portée

```
int x;           // déclarée, valeur indéterminée (danger !)  
int y = 0;       // déclarée et initialisée  
const int MAX = 100; // constante, non modifiable après initialisation
```

Une variable non initialisée contient une valeur indéterminée (« garbage ») : l'utiliser avant affectation est une source classique de bug et de vulnérabilité (lecture de mémoire non initialisée).

3. Opérateurs

- **Arithmétiques** : `+` `-` `*` `/` `%` (le `%` n'existe que pour les entiers).
- **Relationnels** : `==` `!=` `<` `>` `<=` `>=` (renvoient 0 ou 1).
- **Logiques** : `&&` `||` `!`.
- **Bit à bit** : `&` `|` `^` `~` `<<` `>>` — fondamentaux en cryptographie (XOR, masques, décalages) et abordés en profondeur au chapitre Cryptographie.
- **Affectation composée** : `+=` `-=` `*=` `/=` `%=`.
- **Incrémentation/décrémentation** : `++` `--` (pré/post-fixe).

```
int a = 5, b = 3;
int somme = a + b;      // 8
int reste = a % b;     // 2
int masque = a & b;    // ET bit à bit -> 1
```

4. Conversions de type (casting)

```
int i = 7;  
double d = (double) i / 2;    // conversion explicite : 3.5  
int tronque = (int) 3.9;      // conversion explicite : 3
```

Les conversions implicites (entre `int` et `char`, `int` et `float`, etc.) peuvent provoquer des pertes de précision ou des dépassements silencieux : c'est une source fréquente de bugs de sécurité (ex. : troncature d'une taille de buffer calculée en `size_t` vers `int`).

5. Entrées-sorties standard

```
#include <stdio.h>

int main(void) {
    int age;
    printf("Entrez votre âge : ");
    scanf("%d", &age);          // & : adresse de la variable
    printf("Vous avez %d ans.\n", age);
    return 0;
}
```

Spécificateurs de format courants : `%d` (int), `%f` (float/double), `%c` (char), `%s` (chaîne), `%p` (pointeur), `%x` (hexadécimal).

Point de vigilance sécurité : `scanf("%s", ...)` sans limite de taille, ou une chaîne de format contrôlée par l'utilisateur passée directement à `printf`, sont des vulnérabilités classiques (dépassement de tampon, format string). Ce point sera repris en détail dans le cours *Sécurité des applications*.

| À retenir

- Toujours initialiser ses variables.
- Connaître la taille et les limites de chaque type (`limits.h`, `stdint.h`).
- Se méfier des conversions implicites et des chaînes de format non maîtrisées.

Chapitre 3 — Structures de contrôle

Algorithmique et Programmation en C

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Structures conditionnelles

if / else if / else

```
int note = 12;
if (note >= 16) {
    printf("Très bien\n");
} else if (note >= 10) {
    printf("Admis\n");
} else {
    printf("Ajourné\n");
}
```

switch

```
int jour = 3;
switch (jour) {
    case 1: printf("Lundi\n"); break;
    case 2: printf("Mardi\n"); break;
    case 3: printf("Mercredi\n"); break;
    default: printf("Jour invalide\n");
}
```

L'oubli d'un `break` provoque un **fall-through** vers le cas suivant : parfois voulu, souvent source de bug. Toujours documenter un fall-through intentionnel par un commentaire.

Opérateur ternaire

```
int max = (a > b) ? a : b;
```

2. Boucles

while

```
int i = 0;
while (i < 5) {
    printf("%d\n", i);
    i++;
}
```

do ... while

Exécute le corps au moins une fois avant de tester la condition — utile pour valider une saisie utilisateur.

```
int choix;
do {
    printf("Choix (1-3) : ");
    scanf("%d", &choix);
} while (choix < 1 || choix > 3);
```

for

```
for (int i = 0; i < 10; i++) {
    printf("%d ", i);
}
```

3. Rupture de séquence

- `break` : sort immédiatement de la boucle ou du `switch`.
- `continue` : passe directement à l'itération suivante.
- `goto` : à éviter en général (nuît à la lisibilité), sauf cas très spécifiques (sortie d'imbrications profondes, gestion d'erreurs façon « cleanup » en C bas niveau).

4. Boucles infinies et conditions de terminaison

```
for (;;) {  
    // boucle infinie volontaire, sortie via break  
}
```

Une condition de terminaison mal posée (ex. compteur non incrémenté, condition toujours vraie) provoque un déni de service par consommation de ressources — un cas particulier à garder en tête pour le cours *Security by design*.

5. Complexité algorithmique (introduction)

Le nombre d'itérations d'une boucle détermine la complexité temporelle d'un algorithme, notée en notation de Landau (grand O). Exemple : une recherche séquentielle dans un tableau de taille n est en $O(n)$; une recherche dichotomique dans un tableau trié est en $O(\log n)$. Cette notion sera réutilisée pour analyser le coût des algorithmes cryptographiques (chapitre Cryptographie) et des attaques par force brute (chapitre Cryptanalyse).

À retenir

- Choisir la structure adaptée : `for` pour un nombre d'itérations connu, `while`/`do while` pour une condition dynamique.
- Toujours s'assurer qu'une boucle progresse vers sa condition d'arrêt.
- `switch` sans `break` = piège classique.

Chapitre 4 — Fonctions et portée des variables

Algorithmique et Programmation en C

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

| 1. Pourquoi découper en fonctions ?

Une fonction encapsule un traitement réutilisable, améliore la lisibilité, facilite les tests unitaires et limite la portée des erreurs. C'est un principe de base du *Security by design* : plus une fonction est petite et ciblée, plus elle est facile à auditer.

2. Déclaration, prototype, définition

```
// Prototype (souvent dans un .h)
int addition(int a, int b);

int main(void) {
    int resultat = addition(3, 4);
    printf("%d\n", resultat);
    return 0;
}

// Définition
int addition(int a, int b) {
    return a + b;
}
```

Le prototype permet d'appeler une fonction avant sa définition dans le fichier, et de vérifier la cohérence des types passés — le compilateur détecte alors des erreurs qu'une absence de prototype laisserait passer silencieusement.

3. Passage de paramètres

- **Passage par valeur** (défaut en C) : la fonction reçoit une copie ; toute modification est locale.
- **Passage par adresse** (via pointeur) : permet à la fonction de modifier la variable de l'appelant.

```
void incrementer(int *x) {  
    (*x)++;  
}  
  
int main(void) {  
    int n = 5;  
    incrementer(&n);    // n vaut 6 après l'appel  
    return 0;  
}
```

4. Portée (scope) et durée de vie

Type de variable	Portée	Durée de vie
locale	bloc où elle est déclarée	jusqu'à la fin du bloc (pile)
globale	tout le fichier (ou programme si non <code>static</code>)	toute l'exécution
<code>static</code> locale	bloc de déclaration	toute l'exécution, valeur conservée entre appels
paramètre	corps de la fonction	durée de l'appel

```
int compteur_appels(void) {  
    static int n = 0; // initialisée une seule fois  
    n++;  
    return n;  
}
```

Les variables globales facilitent le partage d'état mais nuisent à la prévisibilité du code et compliquent l'analyse de sécurité (effets de bord non locaux) : à limiter au strict nécessaire.

5. Récursivité

```
unsigned long factorielle(unsigned int n) {  
    if (n == 0) return 1;  
    return n * factorielle(n - 1);  
}
```

Chaque appel récursif consomme de la pile ; une récursion non bornée ou trop profonde provoque un **stack overflow**, un cas concret de vulnérabilité par épuisement de ressource.

6. Bonnes pratiques

- Une fonction = une responsabilité claire.
- Toujours valider les paramètres reçus (pointeurs non nuls, tailles cohérentes).
- Documenter précondition/postcondition, y compris qui est responsable de libérer une mémoire éventuellement allouée.

| À retenir

- Le passage par adresse est le mécanisme C pour qu'une fonction modifie les données de l'appelant.
- La portée `static` locale crée un état persistant discret : à utiliser avec parcimonie.
- La récursivité est élégante mais bornée par la taille de la pile.

Chapitre 5 — Tableaux et chaînes de caractères

Algorithmique et Programmation en C

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Tableaux statiques

```
int notes[5] = {12, 15, 8, 17, 10};
int i;
for (i = 0; i < 5; i++) {
    printf("%d\n", notes[i]);
}
```

En C, un tableau `notes[5]` occupe un bloc contigu de mémoire ; `notes[i]` est strictement équivalent à `*(notes + i)`. **Le C n'effectue aucune vérification de borne** : lire ou écrire `notes[5]` ou `notes[-1]` compile et s'exécute, en accédant à de la mémoire hors du tableau. C'est la cause première des débordements de tampon (*buffer overflow*), une des vulnérabilités logicielles les plus exploitées historiquement — approfondie dans *Sécurité des applications*.

2. Tableaux à deux dimensions

```
int matrice[3][4];  
matrice[1][2] = 7;
```

Stockage en mémoire en *row-major order* : les éléments d'une même ligne sont contigus.

3. Chaînes de caractères

En C, une chaîne est un tableau de `char` terminé par l'octet nul `'\0'`.

```
char nom[20] = "Zakaria";
printf("%s\n", nom);
printf("Longueur : %zu\n", strlen(nom)); // 7, sans compter '\0'
```

Fonctions de `string.h`

Fonction	Rôle	Risque
<code>strlen</code>	longueur d'une chaîne	boucle infinie si pas de <code>'\0'</code>
<code>strcpy(dst, src)</code>	copie	aucune vérification de taille → débordement
<code>strncpy(dst, src, n)</code>	copie bornée	ne garantit pas la terminaison par <code>'\0'</code>
<code>strcat(dst, src)</code>	concaténation	même risque que <code>strcpy</code>
<code>strcmp(a, b)</code>	comparaison lexicographique	—
<code>snprintf</code>	formatage borné	alternative sûre recommandée

```
char buffer[10];
strcpy(buffer, "Ceci dépasse dix caractères"); // comportement indéfini : écrase la mémoire adjacente
```

Bonne pratique : préférer systématiquement les variantes bornées (`snprintf`, `strncat` avec vérification manuelle du `'\0'` final) et vérifier la taille des buffers avant toute copie.

4. Tableaux de chaînes

```
const char *jours[] = {"Lundi", "Mardi", "Mercredi"};  
printf("%s\n", jours[1]); // Mardi
```

5. Parcours et algorithmes classiques

- **Recherche** d'un élément (séquentielle, dichotomique sur tableau trié).
- **Tri** : sélection, insertion, bulle (introduction aux algorithmes de tri, complétés par des algorithmes plus efficaces — tri rapide, tri fusion — en exercice de TP).
- **Inversion** d'un tableau en place (technique des deux pointeurs, réutilisée en cryptographie pour l'endianness et le padding).

À retenir

- Un tableau C ne connaît pas sa propre taille à l'exécution : il faut la transmettre séparément.
- Aucune vérification de borne : la responsabilité de la validité des indices incombe entièrement au programmeur.
- Une chaîne C est un tableau de `char` + `'\0'` ; toute fonction qui manipule des chaînes sans vérifier les tailles est un point d'attention sécurité.

Chapitre 6 — Pointeurs et gestion mémoire

Algorithmique et Programmation en C

DR. ZAKARIA SAWADO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Qu'est-ce qu'un pointeur ?

Un pointeur est une variable dont la valeur est une adresse mémoire.

```
int x = 10;
int *p = &x;    // p contient l'adresse de x
printf("%d\n", *p); // déréférencement : affiche 10
*p = 20;        // modifie x via le pointeur : x vaut 20
```

- `&x` : opérateur « adresse de ».
- `*p` : opérateur de déréférencement (« valeur pointée par »).

2. Les quatre zones de la mémoire d'un programme

Zone	Contenu	Durée de vie
Code (text)	instructions du programme	statique
Données statiques	variables globales / <code>static</code>	toute l'exécution
Tas (heap)	allocation dynamique (<code>malloc</code>)	jusqu'à <code>free</code> explicite
Pile (stack)	variables locales, appels de fonction	durée du bloc/de l'appel

Comprendre cette organisation est indispensable pour analyser un débordement de pile, un *use-after-free*, ou une fuite mémoire.

3. Allocation dynamique

```
#include <stdlib.h>

int *tableau = malloc(10 * sizeof(int));
if (tableau == NULL) {
    // échec d'allocation : toujours vérifier
    exit(EXIT_FAILURE);
}
for (int i = 0; i < 10; i++) tableau[i] = i * i;

free(tableau);
tableau = NULL; // évite un « dangling pointer »
```

- `malloc(taille)` : alloue `taille` octets sur le tas, non initialisés.
- `calloc(n, taille)` : alloue et initialise à zéro.
- `realloc(ptr, nouvelle_taille)` : redimensionne un bloc existant.
- `free(ptr)` : libère la mémoire ; **ne pas l'utiliser deux fois** (double free) ni après (*use-after-free*).

4. Classes de vulnérabilités mémoire liées aux pointeurs

Vulnérabilité	Cause
Déréférencement de pointeur nul	oubli de vérifier le retour de <code>malloc</code> ou une valeur non initialisée
Fuite mémoire (<i>memory leak</i>)	<code>malloc</code> sans <code>free</code> correspondant
Use-after-free	utilisation d'un pointeur après <code>free</code>
Double free	<code>free</code> appelé deux fois sur le même pointeur
Dangling pointer	pointeur vers une variable locale dont la portée est terminée
Buffer overflow	écriture au-delà de la mémoire allouée

Ces classes de vulnérabilités constituent le socle du cours *Sécurité des applications* (exploitation) et sont directement liées aux outils d'audit statique/dynamique vus en *Audit organisation et technique*.

5. Arithmétique des pointeurs

```
int tab[5] = {10, 20, 30, 40, 50};  
int *p = tab;      // p pointe sur tab[0]  
printf("%d\n", *(p + 2)); // 30, équivalent à tab[2]  
p++;               // p pointe maintenant sur tab[1]
```

L'arithmétique de pointeur se fait en unités de la taille du type pointé, pas en octets bruts.

6. Pointeurs de fonction (aperçu)

```
int addition(int a, int b) { return a + b; }  
int (*operation)(int, int) = addition;  
printf("%d\n", operation(2, 3)); // 5
```

Utilisés pour construire des callbacks et des tables de dispatch — mécanisme aussi exploité par certaines techniques d'attaque (détournement de flot de contrôle).

| À retenir

- Toujours vérifier le retour de `malloc/calloc/realloc`.
- Toute allocation dynamique doit avoir une libération correspondante, une seule fois.
- Mettre un pointeur à `NULL` après `free` limite les risques de use-after-free.

Chapitre 7 — Structures (Struct) et fichiers

Algorithmique et Programmation en C

DR. ZAKARIA SAWADOGO — ÉCOLE POLYTECHNIQUE DE OUAGADOUGOU

1. Définir une structure

```
struct Utilisateur {  
    char nom[50];  
    int age;  
    float solde;  
};  
  
struct Utilisateur u1 = {"Zakaria", 30, 1500.0f};  
printf("%s a %d ans\n", u1.nom, u1.age);
```

Une structure regroupe des données hétérogènes sous un seul type, brique de base pour modéliser des objets métier (utilisateur, paquet réseau, enregistrement de log, en-tête de fichier).

2. Structures et pointeurs

```
struct Utilisateur *pu = &u1;  
printf("%s\n", pu->nom);    // équivalent à (*pu).nom
```

L'opérateur `->` est l'un des plus fréquents dans du code C réel (listes chaînées, arbres, structures de données de systèmes d'exploitation ou de protocoles réseau).

3. `typedef` et alias de type

```
typedef struct {  
    int x;  
    int y;  
} Point;  
  
Point p = {3, 4};
```

4. Structures de données dynamiques : liste chaînée (introduction)

```
typedef struct Noeud {
    int valeur;
    struct Noeud *suivant;
} Noeud;

Noeud* creer_noeud(int v) {
    Noeud *n = malloc(sizeof(Noeud));
    if (n == NULL) return NULL;
    n->valeur = v;
    n->suivant = NULL;
    return n;
}
```

Cette structure combine tout ce qui a été vu précédemment : allocation dynamique, pointeurs, structures.

5. Manipulation de fichiers

```
#include <stdio.h>

FILE *f = fopen("donnees.txt", "w");
if (f == NULL) {
    perror("Erreur d'ouverture");
    return 1;
}
fprintf(f, "Bonjour %s\n", "Zakaria");
fclose(f);

FILE *lecture = fopen("donnees.txt", "r");
char ligne[100];
while (fgets(ligne, sizeof(ligne), lecture) != NULL) {
    printf("%s", ligne);
}
fclose(lecture);
```

Modes d'ouverture courants : `"r"` (lecture), `"w"` (écriture, écrase), `"a"` (ajout), `"rb"/"wb"` (binaire).

Point de vigilance sécurité : toujours vérifier la valeur de retour de `fopen`, borner la taille lue (`fgets` avec taille explicite plutôt que `gets`, qui est supprimée du standard C11 pour cette raison), et valider tout chemin de fichier construit à partir d'une entrée utilisateur (risque de *path traversal*).

6. Lecture/écriture binaire

```
struct Utilisateur u;  
fwrite(&u, sizeof(struct Utilisateur), 1, f);  
fread(&u, sizeof(struct Utilisateur), 1, f);
```

Utile pour sérialiser des structures de données, mais sensible aux différences d'alignement mémoire et d'endianness entre systèmes — sujet repris en *Cryptographie* (encodage des données avant chiffrement) et *Technologie Blockchain* (sérialisation des transactions).

À retenir

- `struct` modélise des données composites ; combinée à des pointeurs, elle permet de construire des structures de données dynamiques (listes, arbres).
- La gestion de fichiers en C impose une vérification systématique des erreurs (`fopen`, `fread`, `fwrite`) et une fermeture explicite (`fclose`).
- Ce chapitre clôt le socle du langage : les TP suivants consolident ces notions sur des cas pratiques complets.